Reinforcement Learning - Mini Project 2

Amélie Chu 21206329
Alan Tambellini 21203966

November 13, 2026

1 Introduction

In this project, we compare DAgger and RLPD balanced sampling in **SuperTuxKart** (STK). Our implementation is available on GitHub[1]. SuperTuxKart is a game where players compete to finish a number of laps first. In this study, we focus on minimizing the lap time of a single agent, excluding opponent-related interactions. We investigate three learning approaches:

- **Dataset Aggregation (DAgger[2])** is an iterative imitation learning approach that addresses covariate shift through expert labeling. At each iteration, the agent collects observations by interacting with the environment using its current policy, queries the expert for the corresponding actions, and aggregates the labeled data into the existing dataset. The policy is then retrained on the expanded dataset, and the process repeats. Unlike reinforcement learning, DAgger does not require a reward function, its objective is to imitate the expert policy.
- **Behavioral Cloning** is an imitation learning method that trains a policy to reproduce an expert's actions using supervised learning. It corresponds to the initial iteration of DAgger, before iterative data aggregation.
- **Balanced Sampling in Reinforcement Learning with Prior Data (RLPD)[3]** is an off-policy reinforcement learning approach that combines offline and online data. Starting from a pre-collected dataset, the agent gathers new transitions by interacting with the environment. During training, mini-batches are sampled equally from offline and online replay buffers. Unlike DAgger, RLPD does not imitate an expert but learns a policy by maximizing expected cumulative rewards.

2 Evaluation

2.1 Observations & Actions

Our study focuses on a single agent aiming to complete one lap as quickly as possible, ignoring opponent-related items. The model takes the observations in Table 2 as inputs and predicts the actions in Table 1.

While all action fields are retained, some observation fields have variable sizes. Path dimensions depend on the track, while item lists can change during a race (e.g., when a player drops bubblegum, creates item). To ensure fixed-size inputs, we apply padding to these two fields.

Category	Action fields ($n = 7$)
Movement	`acceleration`, `brake`, `steer`, `drift`, `nitro`
Other	`fire`, `rescue`

Table 1: Action space of the kart. Actions in `teal` are boolean, while those in `orange` are continuous.

Category	Observation fields ($n = 24$)
Kart state	`velocity`, `front`, `kart_type`, `max_steer_angle`, `jumping`
Track	`center_path`, `center_path_distance`, `distance_down_track`
Paths	`paths_start`, `paths_end`, `paths_distance`, `paths_width`
Other karts	`karts_position`
Items	`items_position`, `items_type`
Power-ups	`powerup_type`, `powerup_count`, `energy`, `shield_time`
Attachments	`attachment`, `attachment_time_left`
Race state	`phase`, `aux_ticks`, `skeed_factor`

Table 2: Observation space of STK. **Paths** and **Items**' fields are bolded, as their sizes vary across maps.

2.2 Conceptual comparison

Both methods "use" the same expert but learn from different signals as shown in Table 3. DAgger turns imitation into supervised problems: it trains on the states visited by its own policy, labeled by the expert. It can at best match the expert. RLPD is Soft Actor-Critic (SAC[4]) adapted to use prior data online. Contrary to

[1] https://github.com/Franciline/pystk_project [2] https://arxiv.org/abs/1011.0686 [3] https://arxiv.org/abs/2302.02948
[4] https://arxiv.org/abs/1801.01290

DAgger, it never copies the expert and instead maximizes the environment's built-in reward. With a single kart, the ranking term of this reward vanishes, and at each step ($\Delta t = 0.1\,\text{s}$):

$$r_t = \frac{d_t - d_{t-1}}{10} - 0.1 + 10 f_t \qquad (1)$$

$$G = \sum_{t=1}^{T} r_t = \frac{L}{10} - 0.1T + 10 \qquad (2)$$

where d_t is the distance traveled along the track, $f_t \in \{0, 1\}$ indicates lap completion, T is the episode length and L the track length. The progress terms telescope, so on a fixed track, maximizing return is equivalent to minimizing lap time; idling or driving backwards are penalized. RLPD can **theoretically outperform** the expert.

Aspect	DAgger	RLPD
Learning signal	Expert label at every visited state	Reward only
Needs online	An expert queryable at any state	A reward function
Prior data	Expert-labelled, shrinking share of the dataset	Any quality with rewards, half of each batch
Ceiling	The expert	Can exceed the expert
Architecture	Single actor (policy network)	Actor with ensemble of critics and target critics

Table 3: Conceptual differences between DAgger and RLPD. Behavioral Cloning follows the same imitation-learning principle as DAgger but without online training.

DAgger uses cheap, stable supervised learning, while RLPD relies on off-policy bootstrapping, requiring LayerNorm, a critic ensemble and multiple updates per environment step, which is unstable and costly. Since their training procedures differ, a fully matched configuration is difficult. We therefore fix shared settings (task, expert, initial data, online steps, network size and evaluation) while preserving each algorithm's core principles.

2.3 Implementation

We describe the implementation we followed to compare the algorithms on the task. From the available agents in the repository, we select `agent4` (team *The Winners*). Following Section 2.2, we compare both methods on the single task keeping shared settings identical and method-specific as described in Table 4.

2.3.1 Experimental protocol

The kart must complete one lap of the map `abyss`, with a control step of 0.1 s and a maximum of 1000 steps per episode (the expert finishes in about 580 steps).

Both methods use the same observations, actions and hidden layers; RLPD adds critics (Table 4). The `rescue` action is always set to `False` to prevent slow convergence caused by random rescue actions; automatic rescue occurs if the kart falls off the track.

A key challenge was ensuring comparable initial datasets for both methods. RLPD requires initial offline data, collected *"either in the form of high-quality expert demonstrations, or even low-quality but high-coverage exploratory trajectories" [1, p.1]*. We collected 30 expert-driven episodes (28 unique) with ϵ-random actions: 10 trajectories for each $\epsilon \in \{0, 0.1, 0.3\}$, totaling 17,560 transitions.



Figure 1: Race screenshot on `abyss` map, with orange dots overlaid to visualize the next 15 positions with RLPD from Table 5.

For DAgger, the expert labels every visited state, but only the learner drives after the first iteration. We tested two ways to build the initial dataset:

- **Shared dataset (D1):** DAgger loads the same 28 episodes as RLPD, without collecting additional data in the first iteration. Since both executed and expert actions are stored, even ϵ-random steps provide valid expert labels. This ensures identical initial data, although the dataset may be unnecessarily large for DAgger.
- **Smaller dataset (D2):** DAgger generates fewer expert trajectories using the same collection procedure as RLPD, with 3 trajectories per ϵ, totaling 9 episodes. Following iterations remain identical for both approaches.

Both approaches showed similar results (a difference of only 2 steps in their average performance), so we retained the shared dataset for a fairer comparison. Behavioral Cloning is computed as the first iteration of DAgger.

Randomness Issue. With only one kart, running the expert on the same map with different seeds produced nearly identical trajectories, limiting training diversity. During initial experiments, we observed rapid convergence to expert performance and suspected overfitting due to this lack of variability. To address this, we add Gaussian noise to DAgger's actions during collection. RLPD already explores through its stochastic policy.

Setting	DAgger	RLPD
Initial data	D1: 28 episodes (17,560) D2: 9 episodes (∼5,6k)	28 episodes (17,560)
Hidden layers	2×256, ReLU	2×256, ReLU
Critics	–	5, LayerNorm, min of 2
Number of parameters	0.57M	6.3M
Loss	MSE + BCE (1:1)	SAC
Optimiser	Adam, 10^{-3}	Adam, $3 \cdot 10^{-4}$
Batch size	256	128 offline + 128 online
Updates	5 epochs per round, 11 rounds	UTD = 10, 1 actor update per step
Exploration	Gaussian noise, $\sigma = 0.1$ / 0.05	$\alpha_0 = 0.1$, $\bar{\mathcal{H}} = -3$
γ, τ	–	0.99, 0.005

Table 4: Method-specific hyper-parameters. DAgger starts from the RLPD dataset (D1) or from 9 generated episodes (D2), and adds Gaussian noise to the learner's acceleration and steering (std 0.1 and 0.05).
RLPD's TD target takes the minimum of 2 randomly chosen target critics; γ is the discount factor and τ the rate at which the target critics follow the critics. The SAC temperature α, starting at α_0, weights the entropy bonus and is tuned so that the policy's entropy stays near the target $\bar{\mathcal{H}} = -|A|/2 = -3$.
Behavioral Cloning has the same settings as DAgger, with only 1 round for updates and no exploration.

Training & Evaluation. Both methods collect at least 50k online environment steps using their respective policies, completing the current episode when the budget is reached. DAgger trains for 5 epochs before each evaluation, while RLPD performs 10 updates per environment step (UTD[5]= 10). For both, observation normalization is fitted on the initial dataset and then frozen.

Every ∼5k online steps, the deterministic policy is evaluated on 5 validation seeds (10000–10004): 10 evaluations for RLPD and 11 for DAgger, whose first one, before any online step, serves as Behavioral Cloning. The best checkpoint is selected by highest completion rate (lap finished in time), then lowest mean finishing steps. Each experiment is repeated with 3 training seeds (1000, 2000, 3000), also used to seed online episodes. Finally, the **best checkpoints**, along with the expert, are evaluated on 20 held-out race seeds (20000–20019).

Metrics. We report the completion rate and mean finishing steps ± standard deviation over 60 test races per method (3 training seeds × 20 test seeds). Using the same test seeds, we compare each method against the expert through paired per-race differences, averaged across training seeds, with 95% confidence intervals and sign-flip permutation tests. Computational costs are measured by expert queries, gradient updates, and wall-clock training time.

3 Results

3.1 Performance

Policy	Completion	Finishing steps	Δ vs expert [95% CI]	p
Expert (Team4)	20/20	580.25 ± 6.01	–	–
Behavioral Cloning	60/60	601.07 ± 9.24	+20.8 [+17.7, +24.0]	<0.001
DAgger	60/60	585.22 ± 10.09	+5.0 [−0.1, +10.1]	0.06
RLPD	60/60	**566.75** ± 15.24	−13.5 [−18.3, −8.8]	<0.001

Table 5: Final test on the 20 held-out race seeds, over the 3 training seeds (60 races per method; best checkpoints). Δ: mean difference with the expert on the same races (negative is faster), with its 95% confidence interval. p: probability of observing such a large performance difference if the method and expert performed equally well (two-sided sign-flip permutation test); below 0.05, the difference is significant.

All policies finish every race as shown in Table 5. DAgger matches the expert: it is 5 steps (0.9%) slower which is not significant given $p = 0.06$. RLPD is 13.5 steps (2.3%) faster than the expert: their difference is significant with $p < 0.001$, and 18.5 steps faster than DAgger on the same races (95% CI [−24.2, −12.8], $p < 0.001$), on 18 of the 20. On the three training seeds, DAgger's mean finishing steps are almost identical (585.15–585.30), while RLPD's vary more (563.80–569.15).

[5] Update-to-Data ratio

Behavioral Cloning also finishes every race but is 20.8 steps (3.6%) slower than the expert, and its training seeds vary a lot (std of the seeds' mean: 6.0 steps, against 0.08 for DAgger). DAgger's online rounds close this gap (585.2 vs 601.1 steps), as expected from training on the learner's own states rather than only on the expert's.

Figure 2 compares the **final performance distributions** of the three algorithms. Behavioral Cloning exhibits the highest episode lengths and noticeable variability between training seeds. DAgger performs close to the expert, while RLPD achieves the lowest episode lengths. Both DAgger and RLPD show relatively consistent mean performance across training seeds, although RLPD exhibits greater variability across test episodes.

As shown in Figure 3, DAgger closely follows the expert's smooth trajectory (584 vs. 585 steps), with a mean distance of 0.157m. Behavioral Cloning also follows a similar path, with a mean distance of 0.403 m, but fails to avoid bananas near (-25, -100) and (-25, 50), resulting in a slower lap. In contrast, RLPD follows a less smooth trajectory, with a mean distance of 1.410 m from the expert, but completes the lap faster, in 554 steps. This supports our hypothesis from the conceptual comparison section that RLPD can outperform the expert.

RLPD can discover racing lines not demonstrated by the expert, whereas DAgger primarily learns to imitate expert behavior. However, observing RLPD during game-play reveals that it frequently initiates drifts without maintaining them long enough to obtain a boost, whereas the expert never uses drifts.

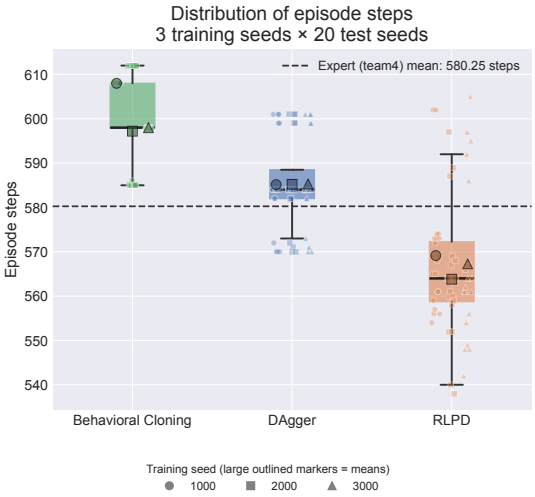


Figure 2: Distribution of episode steps across 3 training seeds, each evaluated on 20 test seeds (60 episodes per algorithm). Boxplots show the median and interquartile range, while individual points represent test episodes and larger markers indicate the mean for each training seed. Lower values indicate faster lap completion.

3.2 Efficiency

Learning speed. Figure 4 shows that DAgger reaches expert-level performance after ∼20k online steps and remains stable, while RLPD reaches it after ∼10k steps and subsequently slightly outperforms the expert. Behavioral Cloning uses no online training. All achieve nearly 100% completion, except RLPD at 30k steps (seed 3000), where one unfinished race causes a spike.

Computational Cost. RLPD has approximately 11 times more parameters than Behavioral Cloning and DAgger (Table 4). Behavioral Cloning requires only ∼345 gradient updates over 5 epochs, compared to ∼9.5k for DAgger and ∼500k critic plus ∼50k actor updates for RLPD. It took around 2.2 hours per run for RLPD, 0.5 hours for DAgger and 3 minutes for Behavioral Cloning.

In terms of expert supervision, RLPD and Behavioral Cloning use only the initial 17,560 offline expert transitions, whereas DAgger also queries the expert at ∼53k visited states online, ∼70k in total (four times more).

Overall, Behavioral Cloning has the lowest computational cost but the poorest performance. DAgger balances computational cost and performance at the expense of additional expert queries, while RLPD achieves the fastest lap times, outperforming the expert, but at the highest computational cost.

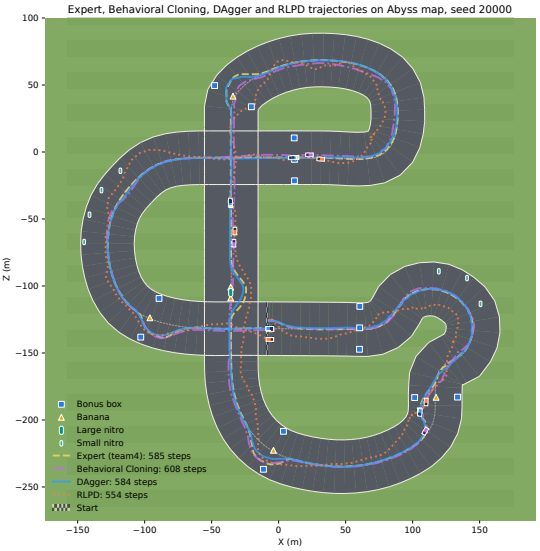


Figure 3: Trajectories on `abyss`, test seed 20000, for the best checkpoint of training seed 1000. Mean distances are computed relative to the expert trajectory. Little colored squares represents karts at different racing point.

4 Multi-maps

4.1 Single map generalization

Since we trained on one map, we tested our best models on an unseen map `minigolf` to which they achieved a 0% completion rate: the models overfit to `abyss`. We therefore repeat the same experiments on 5 training maps (`abyss`, `minigolf`, `volcano_island`, `sandtrack`, `hacienda`) and 3 unseen maps (`snowtuxpeak`, `xr591`, `zengarden`).

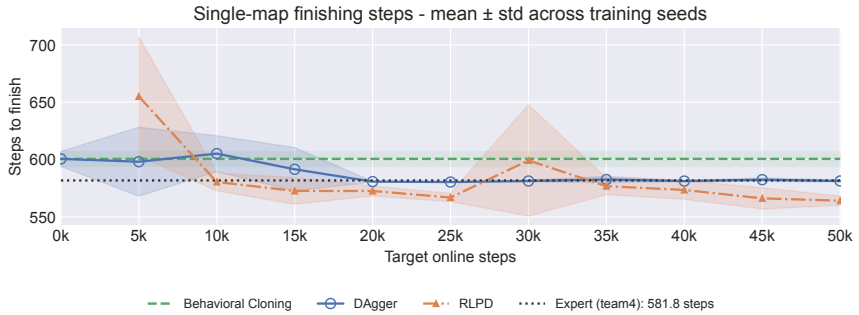


Figure 4: Mean lap completion steps ± standard deviation across training seeds at each evaluation checkpoint.

4.2 Results

For each epsilon, we collect three trajectories per training map, totaling 43 unique trajectories. During training, the actor runs on each map at least once per iteration, always prioritizing maps with the fewest accumulated steps in the current iteration to account for differences in map size. Instead of a fixed 1000-step limit, each map's maximum is set to the expert's highest step count over 5 seeds + 400 (consistent with the previous 1000-step limit for `abyss`, where the expert required ∼580 steps), preventing premature termination and excessively long episodes.



Figure 5: Completion rates on training (25 races) and unseen maps (15 races), with 5 evaluation seeds per map.

Figure 5 shows completion rates on training and unseen maps. On training maps, DAgger and Behavioral Cloning maintain high completion rates (above 80%), while RLPD starts lower but improves with online training. On unseen maps, all methods struggle more. DAgger eventually achieves higher and more stable completion rates despite an earlier drop, whereas RLPD continues to struggle, with lower completion rates and high variability.

5 Conclusion

In this project, we compared Behavioral Cloning, DAgger, and RLPD with balanced sampling in SuperTuxKart, aiming to complete a lap as fast as possible. Despite their different approaches, we established a common protocol and made adjustments to ensure a reasonably fair comparison. We further explored training on multiple maps and evaluating on unseen maps to assess generalization.

Our results show that Behavioral Cloning is the least computationally expensive but achieves the poorest lap-time performance, with limited generalization to unseen maps. DAgger balances computational cost and performance, closely matching the expert while showing better generalization. In contrast, RLPD requires more computational resources but can outperform the expert, although it struggles to generalize to unseen maps. Overall, these results highlight the trade-offs between computational efficiency, expert dependence, performance, and generalization.

AI use statement. We used Opus 5.5 and GPT 6.1 Sol. We questioned an LLM to help us design the experiments, an LLM produced about 40% of the code, and we wrote the report, which an LLM then polished.